

09c — Transactions

Key Concepts

Term	Definition
Transaction	Group of DB operations treated as a single unit of work
BEGIN	Starts a transaction
COMMIT	Permanently saves all changes
ROLLBACK	Undoes all changes since BEGIN
SAVEPOINT	A checkpoint inside a transaction for partial rollback

How It Works

```
BEGIN
  → Run multiple SQL operations
  → If all succeed → COMMIT (changes saved permanently)
  → If any fail → ROLLBACK (all changes undone)
```

Savepoints — Partial Rollback

```
BEGIN
  Operation 1 ☒
  SAVEPOINT checkpoint_1
  Operation 2 ☒ (fails)
  ROLLBACK TO SAVEPOINT checkpoint_1 ← undo only Op 2
  Operation 1 still intact
COMMIT
```

- Use when: complex workflows where one step failing shouldn't undo all prior steps
-

In Entity Framework Core (.NET)

```
using var transaction = context.Database.BeginTransaction();
try {
    // operations
    context.SaveChanges();
    transaction.Commit();
}
```

```
catch {  
    transaction.Rollback();  
}
```

Transaction vs Query

	Query	Transaction
What it is	Single read or write	Wrapper around one or more operations
Guarantee	None	Atomicity — all or nothing

Industry Examples

- **PayPal** — debit sender + credit receiver in one transaction; one fails → both roll back
- **Amazon** — charge card + reserve inventory + create shipment = one atomic transaction
- **Banking batch jobs** — savepoints allow partial recovery without restarting everything

Interview Questions

Question	One-Line Answer
What is a transaction?	Group of DB operations that succeed together or roll back together
COMMIT vs ROLLBACK?	COMMIT saves permanently; ROLLBACK undoes all changes since BEGIN
What is a SAVEPOINT?	Checkpoint inside a transaction for partial rollback without undoing everything
What if you forget to COMMIT?	Transaction stays open, locks held, other transactions may block — risk of deadlock
Transaction vs query?	Query is one operation; transaction wraps multiple with atomicity guarantee